

2. Write a Java program to

a) check whether a given string is a palindrome or not

Type	Input	Expected Output	Explanation
Success	madam	Palindrome	Normal palindrome
Success	level	Palindrome	Odd-length palindrome
Success	hello	Not Palindrome	Non-palindrome
Failure/Edge	a	Palindrome	Single character
Failure/Edge	Madam	Not Palindrome	Case-sensitive check

Algorithm:

1. Start.
2. Create a Scanner object to read input from the user.
3. Read the input string.
4. Initialize an empty string reversed = "".
5. Traverse the input string from the last character to the first.
6. Append each character to reversed.
7. Compare the original string with reversed using the equals() method.
8. If both strings are equal:
 - Display "**Palindrome**".
9. Otherwise:
 - Display "**Not Palindrome**".
10. Close the Scanner.
11. Stop.

Program:

```
import java.util.Scanner;

public class PalindromeCheck {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter a string: ");
        String str = sc.nextLine();

        String reversed = "";

        for (int i = str.length() - 1; i >= 0; i--) {
            reversed += str.charAt(i);
        }

        if (str.equals(reversed)) {
            System.out.println("Palindrome");
        } else {
            System.out.println("Not Palindrome");
        }
    }
}
```

```

    }

    sc.close();
}
}

```

b)sort a list of strings using command-line input

Type	Input	Expected Output	Explanation
Success	apple mango banana	apple banana mango	Alphabetical order
Success	java python c	c java python	Different lengths
Success	dog cat ant	ant cat dog	Normal sorting
Failure/Edge	one	one	Single string
Failure/Edge	Apple apple	Apple apple	Case-sensitive sorting

Algorithm:

1. Start.
2. Check whether any command-line arguments are provided.
3. If no arguments are provided:
 - Display "**No input provided**".
 - Stop the program.
4. If arguments are provided, store them in the args array.
5. Sort the strings in the args array using the Arrays.sort() method.
6. Traverse the sorted array using a loop.
7. Display each string in sorted (alphabetical) order.
8. Stop.

Program:

```

import java.util.Arrays;

public class StringSort {
    public static void main(String[] args) {

        if (args.length == 0) {
            System.out.println("No input provided");
            return;
        }

        Arrays.sort(args);

        for (String str : args) {
            System.out.print(str + " ");
        }
    }
}

```